

GitHub Autopilot

The Complete Project Guide — Architecture, Vision, and
Roadmap for an AI-Powered, Zero-Cost GitHub Repository
Assistant

START-TO-END TECHNICAL DEEP DIVE · V4.2.0

Crafted by **Siddharth Purohit & Shweta Sharma**

Guide compiled June 2026 · Self-hosted Flask GitHub App · ~99 Python modules

Table of Contents

This guide explains the entire project in plain language first, then goes progressively deeper. If you read nothing else, read Parts 1, 2, 14 and 16.

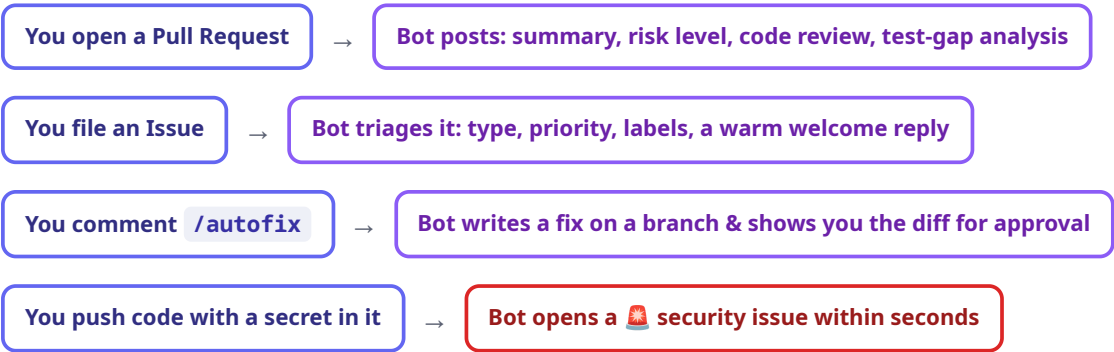
1	What This Project Actually Is (Plain English)	The elevator pitch
2	The Vision — Current & What It Could Become	Where it's headed
3	Glossary — Every Term Defined	Speak the language
4	System Architecture & Component Map	The big picture
5	The Request Lifecycle (4 Phases)	What happens on a webhook
6	The AI Router — Deep Dive	4 brains, 1 interface
7	The Slash-Command System (26 commands)	The user-facing magic
8	The Security Model	7 gates + scanners
9	Reliability & Failure Handling	Never crash, never spam
10	The Codebase Map	Folder-by-folder tour
11	Deployment & Infrastructure	\$0/month on Render
12	The Web Dashboard & Frontend	The demo face
13	Testing	How quality is guarded
14	What Needs Fixing — Bugs & Tech Debt	Honest issues list
15	What Should've Been Done — Roadmap	The gap to greatness
16	Highlights & Final Pointers	The takeaways

1 · What This Project Actually Is

In one sentence: **GitHub Autopilot is a robot teammate you install on your GitHub repositories — it reviews pull requests, triages issues, scans for leaked secrets, and answers slash commands like `/fix` and `/autofix`, all powered by free AI models and running at \$0/month.**

Think of it as a mix between a **code reviewer**, a **project manager**, and a **security guard** that never sleeps. When something happens on your repo (a PR opens, an issue is filed, code is pushed), GitHub sends a message — a *webhook* — to your running copy of this app. The app then decides what to do, asks an AI model for help when needed, and posts a helpful comment back on GitHub.

The day-to-day experience for a user



Why it exists

Most teams want the polish of a tool like a paid AI code-review SaaS, but those cost money per seat and send your code to a third party. This project proves you can get **80% of that value for free**, self-hosted, with full control. Every design decision is bent around three hard constraints:

\$0
MONTHLY COST — ALL FREE TIERS

10s
GITHUB'S WEBHOOK TIMEOUT IT
MUST BEAT

512MB
RAM CEILING ON RENDER FREE TIER

THE CORE INSIGHT
GitHub gives you 10 seconds to acknowledge a webhook or it retries. AI calls take 1–8 seconds and can fail. So the app **acknowledges instantly (HTTP 202), then does the real work in a background thread.** Almost the entire architecture flows from this one constraint.

How big is it, really?

Metric	Value	Meaning
Python modules	~99 files	A genuinely substantial codebase, well-organized into layers
Slash commands	26	From <code>/explain</code> to <code>/rollback</code> to <code>/secfull</code>
AI providers	4	Groq 70B, Groq 8B, Gemini Flash, OpenRouter — with automatic failover
Secret patterns	35+	AWS, GitHub, OpenAI, Anthropic, Azure, GCP, Twilio...

Documentation files	15 markdown docs	Architecture, threat model, deploy guide, AI routing, etc.
Test files	22 test modules	Webhook security, router, secrets, idempotency, confidence...

2 • The Vision

2.1 — The vision as it stands today

The project's stated identity is a **"self-hosted AI assistant for GitHub repositories"** that costs nothing to run. The current, achieved vision is:

- **Autonomous repo hygiene.** PRs get summarized and reviewed; issues get triaged and labelled; commits get linted against Conventional Commits; secrets and vulnerable dependencies get flagged — all without a human lifting a finger.
- **On-demand AI on a comment line.** 26 slash commands turn any GitHub comment box into an AI console.
- **Production-grade reliability on toy-grade infrastructure.** Circuit breakers, idempotency, rate limits, bounded thread pools, and graceful degradation — the kind of engineering normally reserved for paid systems, running on free tiers.
- **Security-first by default.** Fail-closed webhook verification, a 35+ pattern secret scanner, and an autofix engine that refuses to touch sensitive files.

THE ONE-LINE MISSION

"Give every open-source maintainer a tireless, trustworthy AI teammate — without a credit card, without shipping their code to a vendor, and without a single dollar of infrastructure cost."

2.2 — What the vision *could* become

This is where the project has real upside. The bones are strong enough to grow into something much bigger. Here are the most compelling expansions, ranked by ambition:

Vision tier	What it looks like	What it unlocks
NEAR The Closed Learning Loop	The bot already <i>tracks</i> whether its suggestions get accepted. Close the loop: feed acceptance rates back into prompt and provider selection so it measurably gets smarter per-repo over time.	A bot that adapts to each team's taste — the difference between a tool and a teammate.
NEAR Full Codebase Awareness	Embeddings/vector search exist but only feed PR review. Wire them into <code>/fix</code> , <code>/explain</code> , <code>/autofix</code> so every answer is grounded in <i>your</i> actual code, not generic knowledge.	Answers that reference your real functions and conventions — RAG done right.
MID Multi-File, Tested Autofix	Today autofix is single-file and unvalidated. Add a sandboxed test runner so the bot can fix bugs spanning files <i>and prove the fix passes tests</i> before asking for review.	Genuinely trustworthy automated PRs — the holy grail of AI dev tools.
MID The Autopilot Platform	Org-level config with per-repo overrides, a real audit log, a multi-tenant dashboard, and one-click GitHub Marketplace install.	Turns a personal project into a distributable product others can adopt in 60 seconds.
FAR The Repo Operating System	Proactive agent: opens its own PRs to fix flaky tests, bump dependencies, write missing docs, and shepherd releases — supervised by humans via approvals.	The repo maintains itself; humans set direction and approve. The full "autopilot" promise realized.

MY HONEST READ

The project is **~70% of the way to a real product**. The hardest engineering (reliability, security, multi-provider AI) is already done and done well. The gap to greatness is now mostly about *closing loops that are already half-built* (learning, embeddings) and *productizing* (install flow, multi-tenancy). That's a much nicer place to be than having the easy stuff done and the hard stuff missing.

3 · Glossary — Every Term Defined

Keep this page handy. Every concept the project relies on, in plain words.

Term	Definition (in this project's context)
GitHub App	A first-class GitHub integration (not a personal bot account) that gets installed on repos and receives events. It authenticates with a private key and gets short-lived <i>installation tokens</i> . This project IS a GitHub App.
Webhook	An HTTP POST that GitHub fires at your server whenever something happens (PR opened, issue created, push, comment). The entire app is reactive to these.
HMAC-SHA256	A cryptographic signature GitHub attaches to each webhook (header <code>X-Hub-Signature-256</code>) using a shared secret. The app recomputes it to prove the request genuinely came from GitHub and wasn't forged.
Idempotency	The guarantee that processing the same event twice has the same effect as once. GitHub retries webhooks, so the app fingerprints each event (SHA-256) and uses Redis <code>SET NX</code> to skip duplicates.
Circuit Breaker	A safety switch around each AI provider. After 3–5 consecutive failures it "opens" (stops sending traffic for 30–120s), then "half-opens" to test recovery. Prevents hammering a dead service.
LLM Router	The component (<code>app/ai/router.py</code>) that picks <i>which</i> of the 4 AI models to use for a given task, sanitizes input, calls it, parses the result, and falls back if it fails. Handlers never know which model answered.
Task tier	Tasks are classified <code>fast</code> / <code>standard</code> / <code>deep</code> / <code>long</code> . This decides which model gets the job (e.g. cheap 8B for labels, big 70B for code review, Gemini for long context).
Thread Pool	A bounded <code>ThreadPoolExecutor</code> (6 workers, 50-job queue cap) that runs webhook work in the background after the instant ACK. Bounded so a retry-storm can't exhaust RAM.
Hallucination check / Confidence gate	Before posting AI output, the app scores how confident/grounded it is. Low confidence → adds a warning footer or blocks the comment entirely.
Embeddings / Vector search (RAG)	Code files are turned into numeric vectors (via sentence-transformers) and stored in a vector DB (Qdrant/Chroma). Similar code can then be retrieved to give the AI relevant context — "Retrieval-Augmented Generation."
Slash command	A command typed in a GitHub comment, like <code>/fix</code> or <code>/autofix</code> . The comments handler parses it and runs the matching action.
Guardrails	Rules in <code>app/core/guardrails.py</code> that decide whether an automated action (auto-merge, auto-label, title rewrite) is actually allowed to proceed.
Snapshot / Rollback	Before destructive actions, the bot saves a snapshot of repo state to Redis so <code>/rollback</code> can undo bot actions (close created issues, revert title edits, remove labels).
MCP (Model Context Protocol)	A standard that lets IDEs/AI tools call this app's capabilities as "tools." Exposed at the <code>/mcp</code> endpoint.
Fail-closed	A security posture: if something is misconfigured (e.g. missing webhook secret), the app <i>refuses</i> rather than running insecurely. <code>startup_check()</code> won't boot without the secret.
Redis	An in-memory data store used for all shared state: dedup keys, rate limits, AI usage counters, analytics, snapshots. Has an in-memory <i>fake</i> fallback so the app survives Redis going down.

4 · System Architecture & Component Map

The system is built in clean horizontal layers. A request falls down through them like water; each layer has exactly one job.

4.1 — The five layers at a glance

Layer	Lives in	Job
1. Security	<code>server.py</code> , <code>core/webhook_security.py</code> , <code>core/idempotency.py</code>	Verify, rate-limit, dedupe, then ACK in <50ms
2. Dispatch	<code>core/thread_pool.py</code>	Run the work in a bounded background thread pool
3. Handlers	<code>app/handlers/*</code>	One handler per GitHub event type; the business logic
4. AI Router	<code>app/ai/*</code>	Pick a model, sanitize, call, parse, validate, fall back
5. State	Redis + GitHub REST API + scanners	Where everything is read from and written to

4.2 — The full component map



usage · budget snapshots	PRs · releases collaborator perms	dependency CVEs entropy gating
-----------------------------	--------------------------------------	-----------------------------------

WHY THIS SHAPE IS GOOD

Each handler runs in its own thread with its own exception guard — a crash in `push.handle()` can't take down a concurrent `comments.handle()`. And because *all* shared state lives in Redis (not in process memory), the app can be restarted at any moment without losing correctness.

5 · The Request Lifecycle — 4 Phases

This is the single most important flow in the system. Follow one webhook from arrival to GitHub comment.

Phase 1 — Ingress (synchronous, <50ms, GitHub is waiting)

```
GitHub —POST /webhook→ server.py
├── [1] Body > 25 MB? → 413 Payload Too Large X stop
├── [2] IP > 100 req/min? (Redis counter) → 429 Too Many Requests X stop
├── [3] HMAC-SHA256 signature valid? → 401 if mismatch X stop
│   └── (empty secret = 401, NOT bypass — fail closed)
├── [4] JSON parseable? → 400 Bad Request X stop
├── [5] Sender is a bot? → 200 skip (loop guard)
├── [6] Fingerprint already seen? (SET NX) → 200 skip (duplicate)
└── [7] Submit to ThreadPool → 202 ACCEPTED ✓ (done in <50ms)
    └── pool full (>=50 pending)? → 202 + drop + ERROR log
```

WHY ACK BEFORE DOING THE WORK?

If the bot did the AI call first (say 8 seconds) and it failed, GitHub would hit its 10s timeout and *retry* — causing duplicate comments. The correct pattern is: acknowledge instantly, process in the background, and absorb any retries via idempotency.

Phase 2 — Dispatch (async, routing by event type)

```
X-GitHub-Event header → pull_request → pull_request.handle()  issues → issues.handle()
issue_comment → comments.handle()  push → push.handle()  check_run → ci.handle()
```

Phase 3 — Handler execution (the recurring 9-step pattern)

Every handler follows the same disciplined sequence:

```
[1] get_installation_token()  JWT → GitHub → token (cached ~50 min)
[2] load_config()            fetch .ai-repo-manager.yml (5-min cache)
[3] feature-flag check       is this feature enabled in config?
[4] check_command_permission() restricted cmds → GitHub collaborator API
[5] _check_user_rate_limit()  10 commands / hour / user / repo
[6] fetch context            gh_get() issue body, PR files, comments
[7] router.ask()              → Phase 4 (the AI call)
[8] check_response()          hallucination / confidence gate
[9] gh_post()                 post comment / label / PR back to GitHub
```

Phase 4 — AI execution (inside step 7)

```
router.ask(system, user, task)
├── [A] sanitize      strip prompt-injection patterns + char caps
├── [B] TASK_MAP       fast | standard | deep | long
├── [C] select provider circuit state + daily-usage% + task tier
├── [D] provider.ask() HTTP call to LLM, 45s timeout
├── [E] extract JSON   brace-depth scan → strip markdown → {"raw"} fallback
├── [F] breaker update record_success() OR record_failure(reason)
├── [G] track usage     Redis INCR: requests + tokens + cost
└── [H] return (dict, LLMResponse) - on failure, try next provider
```

THE ELEGANCE HERE

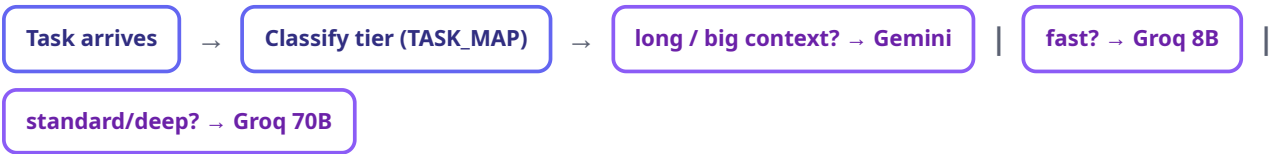
The handler just calls `router.ask("you are a reviewer", code, task="code_review")`. It has no idea whether Groq, Gemini, or OpenRouter answered, whether a fallback fired, or how many tokens it cost. All that complexity is hidden behind one method.

6 · The AI Router — Deep Dive

6.1 — The four-provider chain

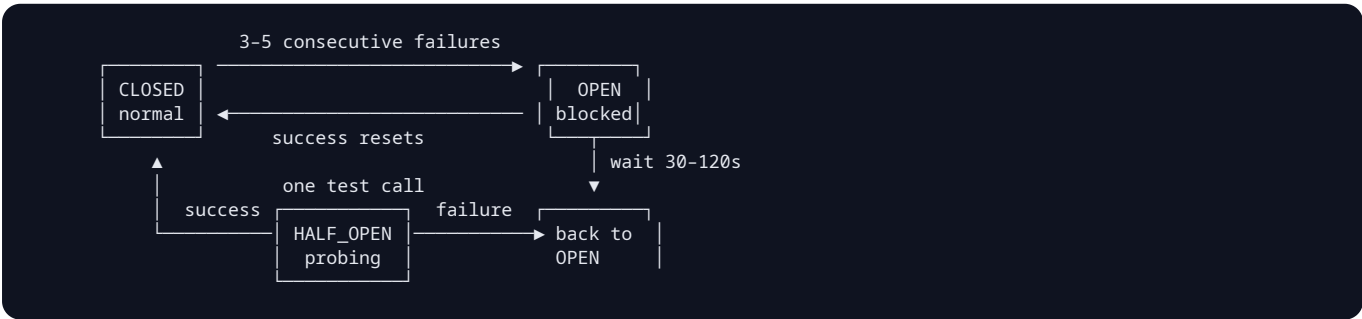
Provider	Model	Role	Free daily limit	Cost/1K
Groq 70B	llama-3.3-70b-versatile	Primary — best quality, standard & deep tasks	5,000 req	\$0.0009
Groq 8B	llama-3.1-8b-instant	Fast tasks (labels, lint, summaries)	12,000 req	\$0.00006
Gemini Flash	gemini-flash	Long context (1M tokens) fallback	1,500 req	\$0 (free)
OpenRouter	various	Emergency last resort	200 req	\$0 (free)

6.2 — How a provider is chosen



Each candidate must pass **two gates** before being picked: its circuit breaker must be available (not OPEN), and its daily usage must be below the threshold (80–85%). If the first choice fails either gate, the router walks down the chain. If *nothing* is available, it raises `AllProvidersDown` — which every handler catches and turns into a graceful "AI temporarily unavailable" message.

6.3 — The circuit breaker state machine



Per-provider tuning (from `circuit_breaker.py`): Groq 70B opens after 3 fails / recovers in 60s; Groq 8B is more tolerant (5 fails / 30s); Gemini 3 fails / 90s; OpenRouter 5 fails / 120s.

6.4 — Defenses baked into every call

- **Prompt-injection sanitization** — user text is scanned for jailbreak patterns ("ignore all previous", "you are now"...) and char-capped before reaching the model.
- **Robust JSON extraction** — LLMs love to wrap JSON in markdown or prose. A brace-depth scanner pulls the JSON out; if it can't, it returns `{"raw": text}` so callers degrade gracefully instead of crashing.
- **Cost & usage tracking** — every call increments Redis counters for requests, tokens, and cost-in-milli-cents per provider per day. This powers the `/budget` command.

7 · The Slash-Command System (26 Commands)

Typed into any GitHub comment. The comments handler extracts the command with word-boundary regex (so `/fix` won't match "prefix"), checks it's enabled, rate-limits the user, checks permissions, then runs it.

Code Quality

Command	What it does	AI?
<code>/fix</code>	Diagnoses an issue and returns root cause + working fix + a test — as a comment (no code change)	✓
<code>/autofix</code>	Actually writes the fix to a new branch, commits it, shows the diff, and asks for <code>/apply</code> confirmation	✓
<code>/apply</code>	Opens a PR from an autofix branch into the default branch HAS BUG	—
<code>/improve</code>	Suggests concrete improvements (perf, security, readability, structure)	✓
<code>/refactor</code>	Proposes refactors with before/after snippets and stated benefits	✓
<code>/perf</code>	Performance analysis: complexity issues, N+1 patterns, quick wins	✓

Understanding

Command	What it does	AI?
<code>/explain</code>	Plain-English explanation of pasted code or the issue	✓
<code>/summarize</code>	Condenses a long issue/PR comment thread	✓
<code>/arch</code>	Architecture review: layer violations, god classes, coupling	✓
<code>/impact</code>	Blast-radius analysis of a PR's file changes	✓
<code>/gaps</code>	Identifies test-coverage gaps with risk ratings	✓
<code>/ci</code>	Analyzes a CI failure (pasted log or fetched from Actions)	✓

Documentation & Release

Command	What it does	AI?
<code>/docs</code>	Generates docstrings, usage examples, README sections	✓
<code>/test</code>	Generates pytest test cases for pasted code	✓
<code>/changelog</code>	Builds a Keep-a-Changelog entry from recent commits	✓
<code>/release</code>	Drafts a GitHub release (auto-bumps version, avoids tag clashes) MAINTAINER	✓
<code>/version</code>	Shows latest tag, release, recent commits	—
<code>/runtests</code>	Triggers a CI test workflow via <code>workflow_dispatch</code>	—

Security, Health & Operations

Command	What it does	AI?
<code>/security</code>	Scans a PR's diff for secrets + vulnerable deps	—
<code>/secfull</code>	Full repository security scan report	—
<code>/health</code>	Repo health score (open issues/PRs, license, description) with a grade	—

<code>/merge</code>	Merges a PR after guardrail checks MAINTAINER	—
<code>/rollback</code>	Two-step confirmed undo of bot actions (with safety snapshot) MAINTAINER	—
<code>/report</code>	Weekly analytics for the repo	—
<code>/budget</code>	AI usage & cost dashboard	—
<code>/notify</code>	Sends a Discord/Slack alert about the issue/PR	—

PERMISSION MODEL

Three commands — `/merge` , `/release` , `/rollback` — are **maintainer-only**. Before running them, the bot calls GitHub's collaborator-permission API and requires `write` , `maintain` , or `admin` . It *fails closed*: any API error denies access. Results are cached 5 minutes per (repo, user).

8 • The Security Model

Security is not bolted on — it's the first thing every request touches and a recurring theme throughout. This is genuinely the project's strongest area.

8.1 — The 7-stage webhook pipeline

#	Gate	Threat it stops	Failure response
1	Size limit (25MB)	Memory-exhaustion via giant payload	413
2	IP rate limit (100/min)	Flooding / DoS from one source	429
3	HMAC-SHA256 verify	Forged webhooks from attackers	401
4	JSON parse	Malformed-payload crashes	400
5	Bot detection	Infinite bot-reply loops	200 skip
6	Replay protection (idempotency)	Duplicate processing of retries	200 skip
7	Bounded dispatch	OOM under retry-storm	202 + drop

8.2 — Secret scanning (35+ patterns)

The `enhanced_secrets.py` scanner detects credentials for AWS, GitHub (PAT/OAuth/App/fine-grained), OpenAI, Anthropic, Azure, GCP, Twilio, SendGrid, Cloudflare, npm, Docker Hub, Heroku, PagerDuty and more. Two clever touches:

- **Entropy gating** — some patterns require *both* a regex match *and* high randomness, drastically cutting false positives.
- **Self-protection** — the example/placeholder strings in the scanner's own source are split-and-joined so GitHub's own secret scanner doesn't flag this file. A nice detail.

8.3 — Autofix safety model

The autofix engine is where AI writes real code, so it's heavily fenced:

- **Blocklist of exact paths** — `server.py` , `.env` , `auth`, `config`, `webhook security`, `Dockerfile`, `requirements.txt` are off-limits.
- **Blocklist of prefixes** — anything under `.github/workflows/` (CI-injection vector), `secrets/` , `keys/` , `certs/` , `app/core/webhook*` .
- **Path-traversal guard** — rejects any path containing `..` or starting with `/.`
- **LLM-steering guard** — if the user named a target file, the user's hint wins over whatever file the LLM suggests (so the model can't redirect a fix to a sensitive file).
- **Truncation guard** — if the rewritten file is <70% of the original length, it's rejected (the model probably cut content off).
- **Human in the loop** — autofix never opens a PR by itself. It commits to a branch, shows the diff, and waits for `/apply` .

THREAT MODEL DOC EXISTS

There's a dedicated [docs/security/threat-model.md](#) enumerating attack vectors and residual risk — the kind of artifact most hackathon projects never produce.

9 • Reliability & Failure Handling

The design principle is: **degrade, never die**. Every external dependency can fail and the bot still responds.

Failure	What happens	What the user sees
One LLM times out	Circuit breaker increments, next provider tried	Slightly slower, still a good answer
All 4 LLMs down	<code>AllProvidersDown</code> caught in handler	"AI temporarily unavailable" message
LLM returns junk JSON	<code>{"raw": text}</code> fallback, logged	Safe fallback content
Redis goes down	<code>_FakeRedis</code> in-memory stub substituted	No visible change (dedup weakened)
GitHub returns 429	Exponential backoff: 2s → 4s → 8s	Delayed but identical result
Invalid config YAML	Warning logged, safe defaults used	Bot runs with defaults
Thread pool full	Event dropped + ERROR log	GitHub auto-retries later
Permission denied	Denial comment posted (not silent)	 Permission Denied @user"

The anti-spam discipline

A naive bot would post a duplicate security issue on every push of the same commit. This project deduplicates aggressively via Redis `SET NX` keys with tuned TTLs:

- Secret findings: 1-hour dedup per unique pattern set
- HIGH-severity dependency alerts: 24-hour dedup per file
- Commit-lint issues: 6-hour dedup
- Only HIGH/CRITICAL deps create issues; LOW/MODERATE are logged silently

10 • The Codebase Map

A folder-by-folder tour so you always know where to look.

```
github-autopilot-main/
├── server.py           ← Flask entry: routes, /webhook, dispatch, dashboard APIs
├── worker.py          ← inactive (kept for future Celery scaling)
├── requirements.txt    ← pinned deps (flask, groq, redis, qdrant, transformers...)
├── render.yaml        ← Render deploy config (web + redis services)
├── Dockerfile / docker-compose.yml
├── .ai-repo-manager.yml ← the per-repo config schema (what users customize)
├── app/
│   ├── handlers/      ← [3] ONE per GitHub event type
│   │   ├── pull_request.py ← PR summary, code review, test gaps, risk
│   │   ├── issues.py    ← triage, labels, priority, welcome
│   │   ├── comments.py  ← the 26 slash commands (1,700 lines)
│   │   ├── push.py      ← secret scan, dep scan, commit lint, indexing
│   │   ├── autofix.py   ← the safe code-writing engine
│   │   └── ci.py / schedule.py ← CI analysis, scheduled tasks
│   ├── ai/            ← [4] the LLM brain
│   │   ├── router.py    ← provider selection + fallback (the core)
│   │   ├── circuit_breaker.py ← per-provider state machines
│   │   ├── hallucination.py ← confidence scoring
│   │   ├── validator.py  ← validate/normalize AI JSON output
│   │   ├── prompt_builder.py ← prompt assembly
│   │   └── providers/    ← groq.py, gemini.py, openrouter.py, base.py
│   ├── core/          ← [1/2/5] the foundation
│   │   ├── webhook_security.py ← HMAC, replay, rate-limit, bot detection
│   │   ├── idempotency.py  ← SHA-256 fingerprint + SET NX
│   │   ├── thread_pool.py  ← bounded dispatch
│   │   ├── redis_client.py ← Redis + _FakeRedis fallback
│   │   ├── config.py       ← .ai-repo-manager.yml loader (RLock cache)
│   │   ├── authorization.py ← collaborator permission checks
│   │   ├── confidence.py   ← confidence gates per action
│   │   ├── guardrails.py   ← auto-merge / auto-label / title rules
│   │   ├── snapshot.py     ← rollback snapshots
│   │   ├── analytics.py    ← per-repo metrics
│   │   ├── learning.py     ← acceptance tracking (loop not yet closed)
│   │   └── metrics·logger·cache·sanitizer·health_check...
│   ├── github/         ← GitHub API client
│   │   ├── client.py     ← gh_get/post/put/delete + retry/backoff
│   │   ├── auth.py       ← JWT → installation token (cached)
│   │   └── rate_limit.py · notifications.py · helpers.py
│   ├── security/        ← scanners
│   │   ├── enhanced_secrets.py ← 35+ patterns + entropy
│   │   └── secrets.py · dependencies.py · scanner.py · licenses.py
│   ├── intelligence/    ← RAG layer (embeddings + retrieval)
│   │   ├── embeddings.py · retrieval.py · summarizer.py
│   ├── mcp/             ← Model Context Protocol server
│   │   └── mcp_server.py  ← (note: server.py imports wrong name — see §14)
│   ├── templates/       ← landing page, dashboard (Jinja2)
│   └── static/           ← CSS + JS (theme, auto-refresh dashboard)
├── docs/                ← 15 markdown docs (architecture, threat model...)
├── tests/               ← 22 test modules
└── archive/             ← retired code (Celery, queues, v1 monolith)
```

11 · Deployment & Infrastructure

11.1 — The \$0 stack

Service	Purpose	Free limit
Render (Web)	Flask + Gunicorn	512MB RAM, 0.5 CPU, 750h/mo
Render (Redis)	State, dedup, cache, analytics	25MB, allkeys-lru
Groq	Llama 70B + 8B	5K + 12K req/day
Gemini Flash	Long-context fallback	1.5K req/day
OpenRouter	Emergency fallback	200 req/day
Qdrant Cloud	Vector DB for code context	1GB

11.2 — How it runs

From `render.yaml` : a single Gunicorn web process with `--workers 1 --threads 8` using the `gthread` worker class, 120s timeout, health-checked at `/ping` . Redis is wired in automatically via `connectionString` . All secrets (`GITHUB_APP_ID` , `GITHUB_PRIVATE_KEY` , `GITHUB_WEBHOOK_SECRET` , API keys) are `sync: false` — set manually in the dashboard, never committed.

A SUBTLE CONFIG TENSION WORTH KNOWING

The architecture docs reason about "2 Gunicorn workers" and a 6-worker thread pool, but `render.yaml` actually deploys **1 Gunicorn worker with 8 threads**. That's actually the *better* choice for free tier (shared Redis state, lower memory), but the docs and config disagree — worth reconciling so future-you isn't confused. Also note the `Dockerfile` hard-codes port 8000 while Render uses `$PORT` ; only the Render path is the real deployment.

11.3 — Health endpoints

Endpoint	Auth	Use
<code>/ping</code>	public	Render liveness probe — minimal, no internals leaked
<code>/health</code>	token-gated	Detailed: Redis, GitHub rate limit, breaker states
<code>/metrics</code>	token-gated	Full metrics snapshot
<code>/api/dashboard</code>	public	Powers the web dashboard (mock data when unconfigured)

12 • The Web Dashboard & Frontend

A polished web UI was added on top of the backend (for demo/hackathon presentation) without changing any backend logic.

- `/` — Landing page: hero, feature cards, architecture flow, command showcase.
- `/dashboard` — Live dashboard: 8 metric cards + a recent-events table, auto-refreshing every 30s by polling `/api/dashboard` and `/api/events/recent`.
- **Design** — dark theme by default, indigo/purple accent, glassmorphism cards, Inter font, Lucide icons, light/dark toggle saved to localStorage.

GRACEFUL WHEN EMPTY

The dashboard API returns realistic *mock* data when env vars/Redis aren't configured, so the UI always looks alive in a demo even before a real GitHub App is connected. Each received webhook is also pushed to a Redis list (`webhook:events:recent` , trimmed to 50, 24h TTL) to feed the events table.

Credits appear in the footer of every page: **"Crafted by Siddharth Purohit & Shweta Sharma."**

13 • Testing

The `tests/` directory holds 22 modules covering the critical paths — a strong signal of engineering maturity for a project this size.

- `test_webhook_security.py` — HMAC, replay
- `test_idempotency.py` — dedup correctness
- `test_router.py` — provider selection
- `test_providers.py` — provider adapters
- `test_confidence.py` — confidence gates
- `test_hallucination.py` — hallucination checks
- `test_secrets.py` / `test_enhanced_secrets.py`
- `test_guardrails.py` — auto-action rules
- `test_autofix.py` — safe-edit logic
- `test_pull_request.py` / `test_issues.py`
- `test_comments.py` / `test_commands_fixed.py`
- `test_ci.py` / `test_push.py`
- `test_analytics.py` / `test_learning.py`
- `test_validator.py` / `test_mcp.py`

RUN THEM

`pytest` (config in `pytest.ini`). The mocking strategy and patterns are documented in `docs/testing/testing-guide.md`. Tip: add a coverage gate in CI so coverage can't silently regress.

14 · What Needs Fixing — Bugs & Tech Debt

An honest list. I verified the first two by reading the code directly — they're real, not guesses.

Confirmed bugs (fix these first)

BUG 1 — THE /MCP POST ENDPOINT IS BROKEN

`server.py:257` does `from app.mcp.server import handle_mcp_request`, but there is no `app/mcp/server.py` — the module is `app/mcp/mcp_server.py`. Any POST to `/mcp` will raise `ModuleNotFoundError` and 500. (The `app/mcp/__init__.py` has a try/except shim, but `server.py` bypasses it by importing the submodule directly.)

Fix: change the import to `from app.mcp.mcp_server import handle_mcp_request` (or import from the package: `from app.mcp import handle_mcp_request`).

BUG 2 — /APPLY DUPLICATE-PR CHECK USES THE WRONG OWNER

`comments.py:412`: `owner = repo.split("/")[1]`. For a repo `"acme/webapp"`, index `[1]` is **"webapp"** (the repo name), not the owner `"acme"`. The subsequent `?head={owner}:{branch}` query is therefore malformed, so the "PR already exists" guard can silently fail to find existing PRs.

Fix: use `repo.split("/")[0]` for the owner.

Documented limitations (by design, but real)

Limitation	Impact	Path forward
No execution sandbox for autofix	Generated code is committed untested; human review mandatory	Sandboxed pytest runner before <code>/apply</code>
Single-file autofix only	Multi-file bugs need manual work	Multi-file patch sets
Blocklist-based injection defense	8-pattern substring match; Unicode tricks can bypass	Classifier-LLM pre-filter
5-min permission cache	Revoked collaborators keep access up to 5 min	Redis cache + pub/sub invalidation
No per-command audit log	Only aggregate daily counts kept	Append-only audit stream in Redis
Learning loop tracks but doesn't act	Acceptance data collected, never used	Feed into prompt/provider selection
Embeddings only feed PR review	<code>/fix</code> , <code>/explain</code> miss codebase context	Wire retrieval into all AI commands

Smaller cleanups

- Docs vs config drift** — architecture docs say "2 Gunicorn workers / 6 thread-pool workers"; `render.yaml` ships 1 worker / 8 threads. Reconcile the numbers.
- Dockerfile port mismatch** — hard-codes 8000; Render uses `$PORT`. Harmless today (Render path wins) but confusing.
- Git history is flat** — three commits all named "first commit". Real history/branching would help future maintenance and make `/rollback`-style features easier to reason about.

- **Two secret scanners coexist** — `secrets.py` (used by push) and `enhanced_secrets.py` (used by comments/ `/security`). Consolidate on the enhanced one everywhere.
- **`.env` is present in the working tree** — confirm it's gitignored and never committed (it is in `.gitignore` , but verify no secrets leaked in history).

15 · What Should've Been Done — & What to Do Next

If I were prioritizing the next month of work, this is the order I'd attack it in.

Sprint 1 — Stop the bleeding (1-2 days)

1. Fix the two confirmed bugs (§14) — both are one-line changes.
2. Add a test for the `/mcp` endpoint and the `/apply` owner logic so they can't silently regress.
3. Reconcile the docs/config worker-count drift; fix the Dockerfile port.

Sprint 2 — Close the half-built loops (1 week)

1. **Wire embeddings into all AI commands.** The retrieval code already exists (`get_relevant_context`); call it from `/fix` , `/explain` , `/improve` , autofix. Biggest quality jump for least effort.
2. **Activate the learning loop.** You already track acceptance — use it: nudge prompt variants or provider choice based on what gets accepted per repo.
3. **Add a real audit log.** Append every command invocation (who, what, when, result) to a Redis stream; surface it in the dashboard.

Sprint 3 — Trustworthy autofix (2 weeks)

1. **Sandboxed test runner.** Before `/apply` , run the repo's tests against the autofix branch in an ephemeral container; only offer the PR if they pass. This is the single feature that would most elevate the product.
2. **Multi-file patches.** Extend autofix to coordinate edits across files.

Sprint 4 — Productize (ongoing)

1. **GitHub Marketplace listing** + a 60-second install flow so others can adopt it.
2. **Org-level config** with per-repo overrides.
3. **Multi-tenant dashboard** with per-repo auth (today's dashboard is single-instance/public).
4. **Replace blocklist injection defense** with a classifier-LLM pre-filter.

THE "SHOULD'VE BEEN DONE" HONEST NOTE

Two things would have paid off if done earlier: (1) **real git history** from day one — flat "first commit" history makes the project hard to audit and undersells the genuine sprint-by-sprint work visible in the code comments; and (2) **wiring features end-to-end before adding new ones** — embeddings and the learning loop were built but never connected, which is effort sitting on the shelf. The good news: connecting them is far easier than building them, and that work is largely done.

16 · Highlights & Final Pointers

★ What's genuinely impressive

- **Real distributed-systems engineering on free infra.** Idempotency via `SET NX`, bounded thread pools, per-provider circuit breakers, graceful Redis degradation — this is production thinking, not hackathon hacking.
- **The 4-provider AI router is the crown jewel.** Task-tier classification, automatic failover, usage-aware selection, cost tracking — all behind a one-line `router.ask()` interface. Excellent abstraction.
- **Security-first throughout.** Fail-closed webhooks, a 35+ pattern scanner with entropy gating and self-protection, and an autofix engine fenced by path blocklists and human-in-the-loop confirmation.
- **Anti-spam discipline.** Tuned dedup TTLs and severity gating mean the bot is helpful, not noisy — the difference between a tool people keep and one they uninstall.
- **Documentation depth.** 15 markdown docs including a threat model and architecture-decision rationale. Rare and valuable.
- **26 commands that actually do useful things**, with a real permission model and per-user rate limiting.

🎯 The three pointers to remember

1 FIX THE 2 CONFIRMED BUGS — THEY'RE ONE-LINERS	2 CONNECT EMBEDDINGS + LEARNING (BUILT, NOT WIRED)	3 SANDBOXED AUTOFIX = THE LEAP TO A REAL PRODUCT
--	---	---

🔗 How to navigate the code as a maintainer

- **Want to change what the bot says?** → the relevant `app/handlers/*.py`.
- **Want to change which AI model is used?** → `app/ai/router.py` (`TASK_MAP` and `_select_provider`).
- **Want to add a slash command?** → add it to `ALL_COMMANDS` + a `_cmd_*` function in `comments.py`, and enable it in `.ai-repo-manager.yml`.
- **Want to change security behavior?** → `app/core/webhook_security.py` and `app/security/*`.
- **Want to understand a decision?** → `docs/architecture/system-architecture.md` explains the "why" behind almost everything.

CLOSING THOUGHT

This is a strong, thoughtfully-engineered project that already does real work reliably. It isn't missing the hard parts — it's missing the *connections* between parts that already exist, plus a couple of one-line bug fixes. You're much closer to "shippable product" than the flat git history suggests. Fix the two bugs, wire up the embeddings and learning loops you already built, add sandboxed autofix, and you have something genuinely worth putting on the GitHub Marketplace. Best of luck, brother — you've built something real here. 🚀